

RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105



AI23331 Fundamentals of Machine Learning

Laboratory Record Note Book

Name : MUHAMMED NAVEED KHAN MJ.

Year / Branch / Section : IIIrd Year CSE - C

University Register No. : 2116230701200

College Roll No. : 230701200

Semester : VIth.

Academic Year : 2023-2027

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AI23331 Fundamentals of Machine Learning

NAME	MUHAMMED NAVEED KHAN MJ
ROLL NO	230701200
YEAR	IIIrd Year
SEC	C



RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institution

BONAFIDE CERTIFICATE

Name:

Academic Year: **Semester:** **Branch:**

Register No.

Certified that this is the bonafide record of work done by the above student in

the

Laboratory

during the academic year 2025 – 2026.

Signature of Faculty in-charge

Submitted for the Practical Examination held on

Internal Examiner

External Examiner

INDEX

S.NO	Experiment	Date	GitHub
1a	A python program to implement Data Cleaning.	7/2/2026	
1b	A python program to implement univariate regression, bivariate regression and multivariate regression.	21/2/2026	
2	A python program to implement Simple linear regression using Least Square Method.	28/2/2026	
3	A python program to implement logistic model.	28/2/2026	
4	A python program to implement single layer perceptron.	7/3/2026	
5	A python program to implement multi-layer perceptron with back propagation.	7/3/2026	
6	A python program to do face recognition using SVM classifier.	14/3/2026	
7	A python program to implement decision tree.	14/3/2026	
8	A python program to implement boosting.	11/4/2026	
9a	A python program to implement KNN.	11/4/2026	
9b	A python program to implement K-means.	11/4/2026	
10	A python program to implement dimensionality reduction – PCA.	18/4/2026	

Experiment 1A: Univariate, Bivariate and Multivariate Regression

Aim

To implement univariate, bivariate and multivariate linear regression using the Iris dataset and visualize the regression results.

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import
LinearRegression from sklearn.metrics import
r2_score
from mpl_toolkits.mplot3d import
Axes3D from sklearn.datasets import
load_iris

iris = load_iris()
data = pd.DataFrame(iris.data, columns=iris.feature_names)
data['target'] = iris.target

y = data['sepal length (cm)'] -----

# UNIVARIATE REGRESSION
X_uni = data[['petal length (cm)']]
model_uni = LinearRegression()
model_uni.fit(X_uni, y)
y_pred_uni = model_uni.predict(X_uni)
r2_uni = r2_score(y, y_pred_uni)

# Plot Univariate
plt.figure()
plt.scatter(X_uni, y, color='blue', label='Actual Data')
plt.plot(X_uni, y_pred_uni, color='red', label='Regression Line')
plt.xlabel('Petal Length (cm)')
plt.ylabel('Sepal Length (cm)')
plt.title('Univariate Regression')
plt.legend()
plt.show()

print("UNIVARIATE REGRESSION")
print("Coefficient:",
model_uni.coef_) print("Intercept:",
model_uni.intercept_) -----
print("R-squared:", r2_uni)

# BIVARIATE REGRESSION
X_bi = data[['petal length (cm)', 'petal width (cm)']]
model_bi = LinearRegression()
model_bi.fit(X_bi, y)
y_pred_bi =
model_bi.predict(X_bi) r2_bi =
r2_score(y, y_pred_bi)

# Plot
Bivariate fig =
plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X_bi.iloc[:, 0], X_bi.iloc[:, 1], y,
color='blue') ax.set_xlabel('Petal Length (cm)')
ax.set_ylabel('Petal Width (cm)')
ax.set_zlabel('Sepal Length (cm)')
ax.set_title('Bivariate Regression')
```

```
plt.show()
```

```

print("\nBIVARIATE REGRESSION")
print("Coefficients:", model_bi.coef_)
print("Intercept:", model_bi.intercept_)
print("R-squared:", r2_bi)

# ----- MULTIVARIATE REGRESSION -----
X_multi = data.drop(columns=['target'])
model_multi = LinearRegression()
model_multi.fit(X_multi, y)
y_pred_multi = model_multi.predict(X_multi)
r2_multi = r2_score(y, y_pred_multi)

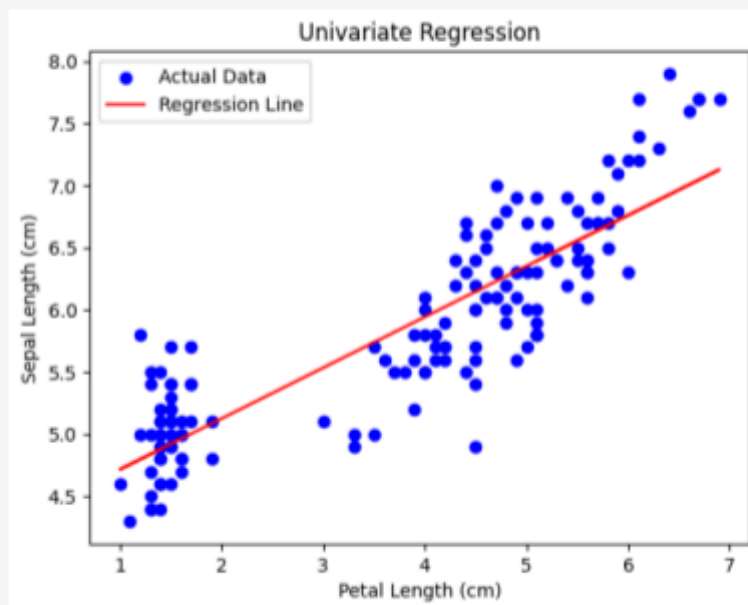
# Plot Multivariate
plt.figure()
plt.scatter(y, y_pred_multi, color='green')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.title('Multivariate Regression')
plt.show()

print("\nMULTIVARIATE REGRESSION")
print("Coefficients:", model_multi.coef_)
print("Intercept:", model_multi.intercept_)
print("R-squared:", r2_multi)

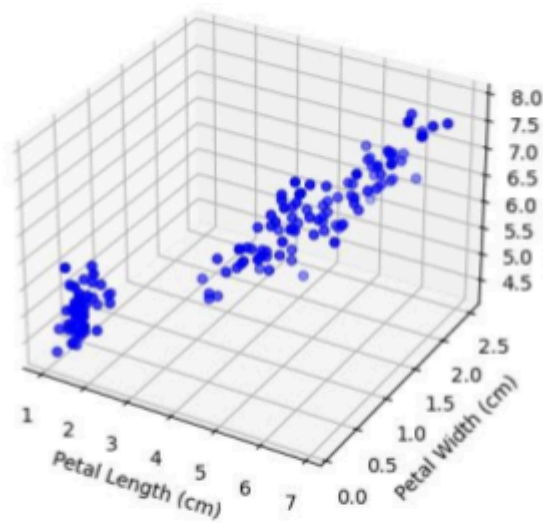
```

Result / Output:

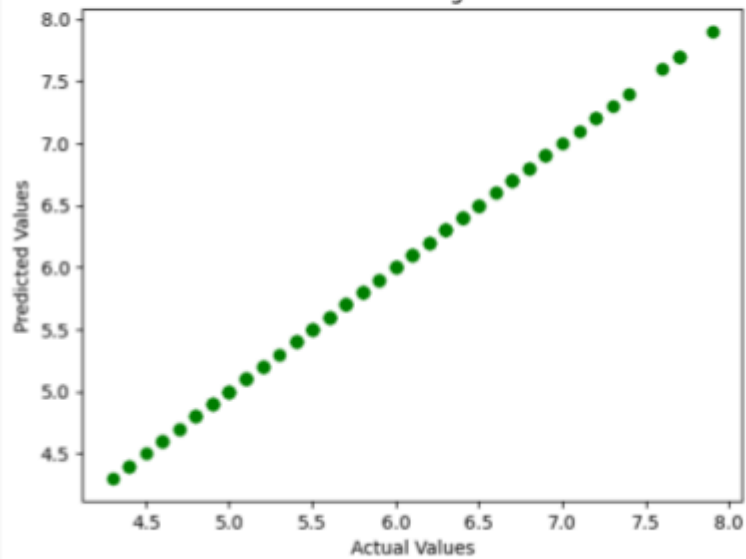
Output:



Bivariate Regression



Multivariate Regression



Experiment 1B: Data Cleaning and Preprocessing

Aim

To perform data cleaning tasks including handling missing values, detecting outliers using IQR, computing correlation, and normalizing/standardizing features.

Program:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("Ex1b.csv")

# Explore the dataset
df.head()
df.tail()
df.info()
df.isnull().sum()
df.describe()

# Boxplot for outlier detection
numeric_df = df.select_dtypes(include='number')
fig, axs = plt.subplots(len(numeric_df.columns), 1, figsize=(7, 18), dpi=95)
for i, col in enumerate(numeric_df.columns):
    axs[i].boxplot(numeric_df[col].dropna(), vert=False)
    axs[i].set_ylabel(col)
plt.tight_layout()
plt.show()

# Remove outliers from Cholesterol using IQR
q1, q3 = np.percentile(df["Cholesterol"].dropna(), [25, 75])
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
clean_df = df[(df["Cholesterol"] >= lower) & (df["Cholesterol"] <= upper)]

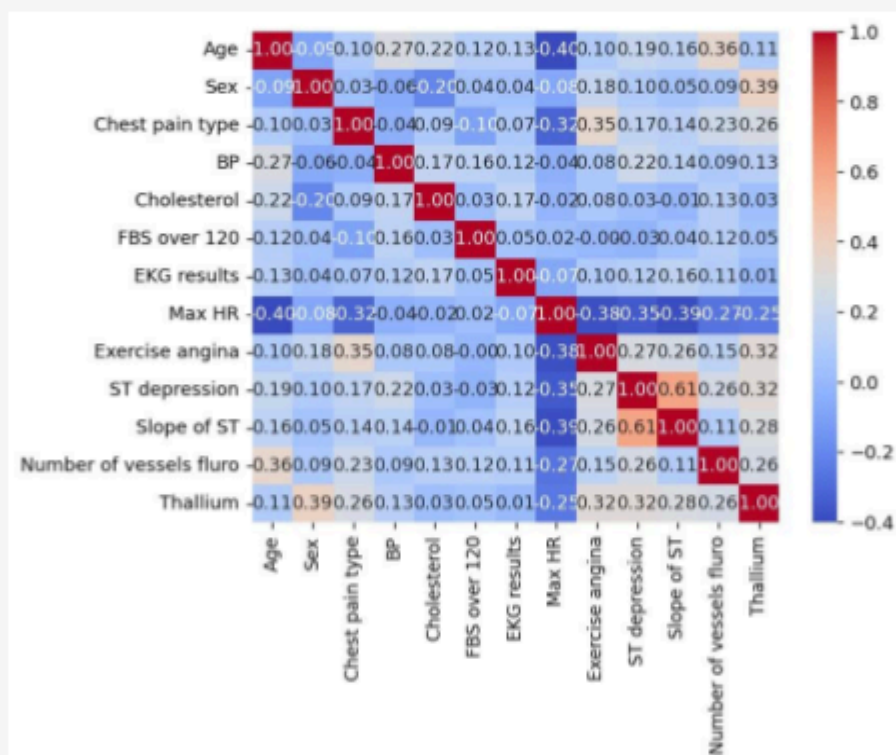
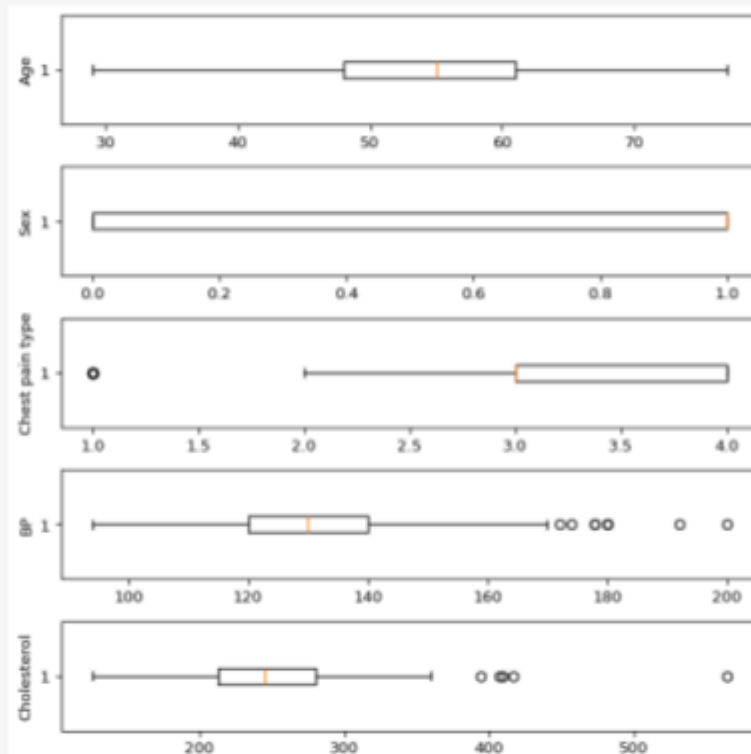
# Correlation heatmap
corr = df.select_dtypes(include='number').corr()
plt.figure(dpi=130)
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm')
plt.show()

# Min-Max Normalization
X = df.select_dtypes(include='number')
X_normalized = MinMaxScaler().fit_transform(X)
print(X_normalized[:5])

# Standardization
X_standardized = StandardScaler().fit_transform(X)
print(X_standardized[:5])
```

Result / Output:

Output:



Experiment 2: Simple Linear Regression using Least Square Method

Aim

To implement simple linear regression using the least square method and evaluate model performance using R2 score and Mean Squared Error.

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

df = pd.read_csv("Ex2.csv")
df.head()

x = df[['sqft_living']]
y = df['price']

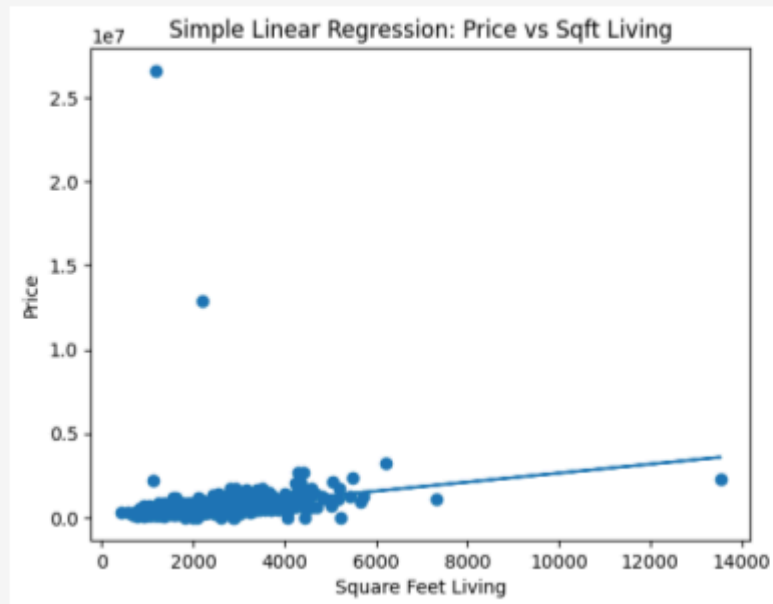
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

print("Slope (m):", model.coef_[0])
print("Intercept (c):", model.intercept_)
print("R2 Score:", r2_score(y_test, y_pred))
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))

plt.scatter(x_test, y_test)
plt.plot(x_test, y_pred)
plt.xlabel("Square Feet Living")
plt.ylabel("Price")
plt.title("Simple Linear Regression: Price vs Sqft Living")
plt.show()
```

Result / Output:**Output:**

Experiment 3: Logistic Regression

Aim

To implement logistic regression for multi-class classification on the Digits dataset and evaluate the model using accuracy score and classification report.

Program:

```
from sklearn.model_selection import train_test_split
from sklearn import datasets
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

digits = datasets.load_digits()
x = digits.data
y = digits.target

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)

model = LogisticRegression(max_iter=10000)
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

accuracy = accuracy_score(y_test, y_pred)
print("Logistic Regression Accuracy:", accuracy)

print("Classification Report: ")
print(classification_report(y_test, y_pred))
```

Result / Output:

Output:

Logistic Regression Accuracy: 0.975

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	33
1	0.97	1.00	0.98	28
2	1.00	1.00	1.00	33
3	0.97	0.97	0.97	34
4	1.00	0.98	0.99	46

5	0.92	0.96	0.94	47
6	0.97	0.97	0.97	35
7	1.00	0.97	0.99	34
8	0.97	0.97	0.97	30
9	0.97	0.95	0.96	40
accuracy			0.97	360
macro avg	0.98	0.98	0.98	360
weighted avg	0.98	0.97	0.98	360

Experiment 4: Single Layer Perceptron (SLP)

Aim

To implement a Single Layer Perceptron (SLP) from scratch to learn the AND gate logic using the step activation function and weight update rule.

Program:

```
import numpy as np

# Input dataset (AND gate)
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

# Target output
y = np.array([0, 0, 0, 1])

# Initialize weights and bias
weights = np.zeros(X.shape[1])
bias = 0
learning_rate = 0.2
epochs = 5

# Activation function (Step Function)
def step_function(x):
    if x >= 0:
        return 1
    else:
        return 0

# Training the perceptron
for epoch in range(epochs):
    print(f"\nEpoch {epoch+1}")
    for i in range(len(X)):
        linear_output = np.dot(X[i], weights) + bias
        y_pred = step_function(linear_output)
        error = y[i] - y_pred
        weights = weights + learning_rate * error * X[i]
        bias = bias + learning_rate * error
    print(f"Input: {X[i]}, Predicted: {y_pred}, Error: {error}")

print("\nFinal Weights:", weights)
print("Final Bias:", bias)

print("\nTesting Perceptron")
for i in range(len(X)):
    linear_output = np.dot(X[i], weights) + bias
    y_pred = step_function(linear_output)
    print(f"Input: {X[i]}, Output: {y_pred}")
```

Result / Output:

Output:

Epoch 1

Input: [0 0], Predicted: 1, Error: -1

Input: [0 1], Predicted: 0, Error: 0

Input: [1 0], Predicted: 0, Error: 0

Input: [1 1], Predicted: 0, Error: 1

Epoch 2

Input: [0 0], Predicted: 1, Error: -1

Input: [0 1], Predicted: 1, Error: -1

Input: [1 0], Predicted: 0, Error: 0

Input: [1 1], Predicted: 0, Error: 1

Epoch 3

Input: [0 0], Predicted: 0, Error: 0

Input: [0 1], Predicted: 1, Error: -1

Input: [1 0], Predicted: 1, Error: -1

Input: [1 1], Predicted: 0, Error: 1

Epoch 4

Input: [0 0], Predicted: 0, Error: 0

Input: [0 1], Predicted: 0, Error: 0

Input: [1 0], Predicted: 0, Error: 0

Input: [1 1], Predicted: 1, Error: 0

Epoch 5

Input: [0 0], Predicted: 0, Error: 0

Input: [0 1], Predicted: 0, Error: 0

Input: [1 0], Predicted: 0, Error: 0

Input: [1 1], Predicted: 1, Error: 0

Final Weights: [0.4 0.2]

Final Bias: -0.40000000000000001

Testing Perceptron

Input: [0 0], Output: 0

Input: [0 1], Output: 0

Input: [1 0], Output: 0

Input: [1 1], Output: 1

Experiment 5: Multi-Layer Perceptron (MLP) with Backpropagation

Aim

To implement a Multi-Layer Perceptron (MLP) with backpropagation from scratch to solve the XOR problem using the sigmoid activation function.

Program:

```
import numpy as np

# Input dataset (XOR gate)
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

# Target output
y = np.array([[0], [1], [1], [0]])

# Sigmoid activation
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Initialize weights randomly
np.random.seed(1)
input_layer_neurons = 2
hidden_layer_neurons = 2
output_neurons = 1

W1 = np.random.uniform(size=(input_layer_neurons,
                               hidden_layer_neurons))
W2 = np.random.uniform(size=(hidden_layer_neurons,
                               output_neurons))

learning_rate = 0.1
epochs = 10000

for i in range(epochs):
    # Forward Propagation
    hidden_input = np.dot(X, W1)
    hidden_output = sigmoid(hidden_input)
    final_input = np.dot(hidden_output, W2)
    predicted_output = sigmoid(final_input)

    # Error
    error = y - predicted_output

    # Backpropagation
    d_predicted_output = error * sigmoid_derivative(predicted_output)
    error_hidden_layer = d_predicted_output.dot(W2.T)
    d_hidden_layer = error_hidden_layer * sigmoid_derivative(hidden_output)

    # Updating weights
    W2 += hidden_output.T.dot(d_predicted_output) * learning_rate
    W1 += X.T.dot(d_hidden_layer) * learning_rate
```

```
print("Predicted Output:")  
print(predicted_output)
```

Result / Output:

Output:

Predicted Output:

[[0.25840124]

0.69139438]

0.69130801]

0.39872979]]

Experiment 6: Face Recognition using SVM Classifier

Aim

To implement a real-time face recognition system using face encodings and a Support Vector Machine (SVM) classifier trained on a custom dataset.

Program:

```
import os
import cv2
import
face_recognition from
sklearn import svm

# Path to dataset folder
dataset_path =
r"C:\path\to\dataset"

encodings = []
names = []

# Read all person folders
for person in os.listdir(dataset_path):
    person_path = os.path.join(dataset_path,
    person) if not os.path.isdir(person_path):
        continue

    for image_name in os.listdir(person_path):
        if not image_name.lower().endswith(('.jpg', '.jpeg',
        '.png')): continue
        image_path = os.path.join(person_path, image_name)

        try:
            image = face_recognition.load_image_file(image_path)
            face_locations =
            face_recognition.face_locations(image) if
            len(face_locations) != 1:
                continue
            encoding = face_recognition.face_encodings(image, face_locations)[0]
            encodings.append(encoding)
            names.append(person)
        except:
            continue

# Train SVM classifier
clf = svm.SVC(gamma='scale', probability=True)
clf.fit(encodings, names)
print("Training Complete!")

# Start webcam
video_capture =
cv2.VideoCapture(0) while True:
    ret, frame =
    video_capture.read() if not
    ret:
        break

    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    face_locations = face_recognition.face_locations(rgb_frame)
    face_encodings = face_recognition.face_encodings(rgb_frame, face_locations)

    for (top, right, bottom, left), face_encoding in zip(face_locations,
    face_encodings): name = "Unknown"
        try:
            prediction = clf.predict([face_encoding])
```

```
probabilities = clf.predict_proba([face_encoding]) confidence
                = max(probabilities[0])
                if confidence > 0.7:
                    name = prediction[0]
except:
    pass

cv2.rectangle(frame, (left, top), (right, bottom), (0, 255, 0), 2) cv2.putText(frame,
    name, (left, top - 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)

cv2.imshow("Face Recognition using SVM", frame) if
    cv2.waitKey(1) == 27:
break

video_capture.release()
cv2.destroyAllWindows()
```

Result / Output:

Output:

Experiment 7: Decision Tree Classifier

Aim

To implement a Decision Tree classifier using the entropy criterion on the Iris dataset and visualize the resulting decision tree.

Program:

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# Load Iris dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Create and train Decision Tree model
model = DecisionTreeClassifier(criterion='entropy')
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicted values:", y_pred)
print("Actual values      :",
      y_test)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Visualize the Tree
plt.figure(figsize=(12, 8))
plot_tree(model, feature_names=iris.feature_names,
          class_names=iris.target_names, filled=True)
plt.show()
```

Result / Output:

Output:

Predicted values: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0
0 0 0 1 0 0 2 1 0 0 0 2 1 1 0 0]

Actual values: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0
0 0 1 0 0 2 1 0 0 0 2 1 1 0 0]

Accuracy: 1.0



Experiment 8A: Boosting – AdaBoost

Aim

To implement the AdaBoost ensemble classifier on the Wine dataset and evaluate performance using accuracy score and classification report.

Program:

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.datasets import load_wine

data = load_wine()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Create AdaBoost model
model = AdaBoostClassifier(
    n_estimators=50,
    learning_rate=1.0,
    random_state=42
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Actual Values : ", y_test)
print("Predicted Values:", y_pred)

accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Result / Output:

Output:

Actual Values : [0 0 2 0 1 0 1 2 1 2 0 2 0 1 0 1 1 1 0 1 0 1 1 2 2 2 1 1 1
0 0 1 2 0 0 0 2 2 1 2 0 1 1 1 2 0 1 1 2 0 1 0 0 2]

Predicted Values: [0 0 2 0 1 0 1 2 1 2 0 1 0 1 0 1 1 1 0 1 0 1 1 2 2 2 1 1 1
0 0 1 2 0 0 0 2 2 1 2 0 1 1 1 2 0 1 1 2 0 1 0 0 2]

Accuracy: 0.9814814814814815

Classification Report:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	1.00	1.00	1.00	19
1	0.95	1.00	0.98	21
2	1.00	0.93	0.96	14
accuracy			0.98	54
macro avg	0.98	0.98	0.98	54
weighted avg	0.98	0.98	0.98	54

Experiment 9A: K-Nearest Neighbors (KNN)

Aim

To implement the K-Nearest Neighbors classifier on a custom dataset, evaluate using confusion matrix and accuracy, with feature scaling applied.

Program:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score

df = pd.read_csv("knn-dataset.csv")

X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0
)

# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create KNN model
knn = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)

print("Predicted values:")
print(y_pred)
print("\nActual values:")
print(y_test)
print("\nConfusion Matrix:")
print(cm)
print("\nAccuracy:", accuracy)
```

Result / Output:

Output:

```
[0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 0 0 0
0 0 1 0 0 0 0 1 0 0 1 0 1 1 0 0 0 1 1 0 0 1 0 0 1 0 1 0 1 0 0 0 0 1 0 0 1
0 0 0 0 1 1 1 0 0 0 1 1 0 1 1 0 0 1 0 0 0 1 0 1 1 1]
```

Confusion Matrix:

```
[[64  4]
```

```
 [ 3 29]]
```

Accuracy: 0.93

Experiment 9B: K-Means Clustering

Aim

To implement K-Means clustering and determine the optimal number of clusters using the Silhouette Score method.

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

df = pd.read_csv("kmeans-dataset.csv")

# Select Annual Income and Spending Score columns
X = df.iloc[:, [3, 4]].values

# Compute silhouette scores for different values of k
silhouette_scores = []

for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, init='k-means++', random_state=0)
    labels = kmeans.fit_predict(X)
    score = silhouette_score(X, labels)
    silhouette_scores.append(score)
    print(f"k = {k}, Silhouette Score = {score:.4f}")

# Plot the silhouette scores
plt.plot(range(2, 11), silhouette_scores, marker='o')
plt.title("Silhouette Method for Optimal k")
plt.xlabel("Number of Clusters")
plt.ylabel("Silhouette Score")
plt.show()

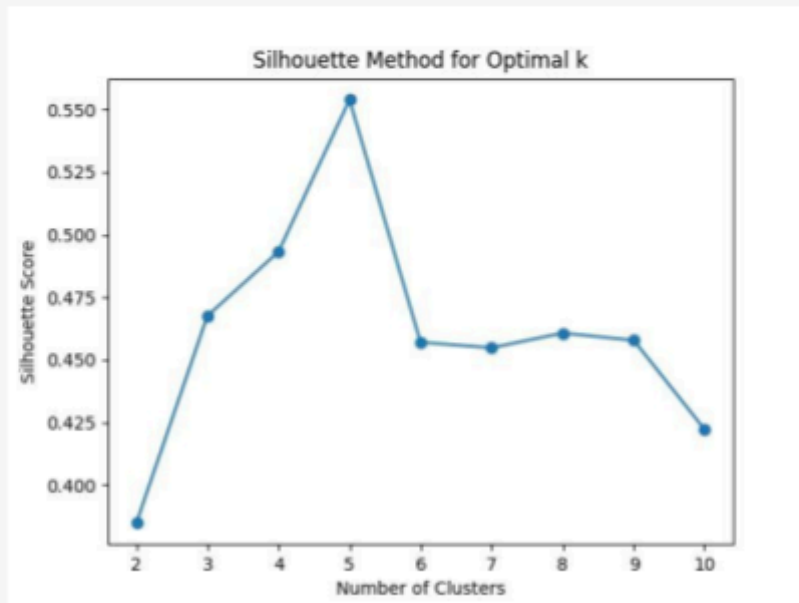
best_k = range(2, 11)[silhouette_scores.index(max(silhouette_scores))]
print("\nBest number of clusters:", best_k)
```

Result / Output:**Output:**

```
k = 2, Silhouette Score = 0.3849
k = 3, Silhouette Score = 0.4676
k = 4, Silhouette Score = 0.4932
k = 5, Silhouette Score = 0.5539
k = 6, Silhouette Score = 0.4571
k = 7, Silhouette Score = 0.4549
k = 8, Silhouette Score = 0.4607
```

$k = 9$, Silhouette Score = 0.4579

$k = 10$, Silhouette Score = 0.4222



Experiment 10: Dimensionality Reduction – PCA

Aim

To apply Principal Component Analysis (PCA) on the Iris dataset to reduce dimensionality from 4 features to 2 principal components and analyze the explained variance.

Program:

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import pandas as pd

iris = load_iris()
X = iris.data
y = iris.target

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply PCA to reduce from 4 features to 2 principal components
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Create a DataFrame to view the result
df = pd.DataFrame(
    data=X_pca,
    columns=['Principal Component 1', 'Principal Component 2']
)
df['Species'] = y

print(df.head())

print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)
```

Result / Output:**Output:**

	Principal Component 1	Principal Component 2	Species
0	-2.264703	0.480027	0
1	-2.080961	-0.674134	0
2	-2.364229	-0.341908	0
3	-2.299384	-0.597395	0
4	-2.389842	0.646835	0

